

予選 解説

2006年1月15日・情報オリンピック日本委員会

第5回日本情報オリンピック 予選

問題 1 カードゲーム (Card Game)

配点 20点 競技時間 180分 採点用入力 5組

編集注：原資料には問題名が記載されていないため、本 PDF の日本語表題と英訳は問題内容に基づく便宜的な見出しです。解説本文は原文の表記を維持しています。

解説

1 番の問題であるから、プログラムの初歩さえ知っていれば解ける問題を出題した。特にむずかしいことは何もないので、この問題で確実に 20 点を取ってほしい。

プレイヤー A, B の得点を保持する変数 A, B を用意し、最初はそれぞれ 0 に初期化しておく。入力ファイルからカードを 2 枚読み込む（読み込んだ A のカードの数字を a とし、B のカードの数字を b とする）たびに、

1. $a > b$ だったら、A に a を加え、
2. $a < b$ だったら、B に b を加え、
3. $a = b$ だったら、A にも B にも a を加える

原文：https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/2006-yo-t1-review.html

問題2 文字変換 (Character Conversion)

配点 20点

競技時間 180分

採点用入力 5組

解説

この問題も 1 番と同様に易しい問題である。文字を扱うことと、配列を使う必要があることくらいが 1 番と違う点である。

まず、入力ファイルの 2 行目から $n + 1$ 行目までのデータを読んで、変換表を 2 次元配列に格納する。次いで、変換する文字を 1 文字ずつ読み込み、その文字を変換表から検索し、対応する文字を出力する。変換表は大きさが小さいので、このときの検索方法は単純に順番に比較する方法で十分である。

このような誰でも思いつくような単純な探索法にも名前が付いていて、線形探索 (linear search) と呼ばれている。それに対し、大きさの順に並んでいるデータ集合からあるデータ x を探索するとき、全体の中央のデータ c と x を比較し、

1. $x = c$ なら、見つかったので終了。
2. $x < c$ なら、 c より小さい方の半分のデータの中で同じ方法（再帰呼び出し）で x を探索する。
3. $x > c$ なら、 c より大きい方の半分のデータの中で同じ方法（再帰呼び出し）で x を探索する。

という探索法を 2 分探索 (binary search) という（上記の 2., 3. では「再帰呼び出し」と書いたが、もちろん再帰を使わなくてもまったく同じことができる）。データの個数が n であるとき、線形探索法では探索にかかる時間が n に比例するのに対し、2 分探索法では探索にかかる時間は悪くても高々 $\log n$ に比例する時間以下であるので、非常に高速である。

原文：https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/2006-yo-t2-review.html

問題3 サイコロ (Dice)

配点 20点

競技時間 180分

採点用入力 5組

解説

この問題はサイコロの各面が指示された操作により刻々と変化していく状態をトレース（追跡）していく、空間認識とシミュレーションの問題である。シミュレーション方法は大きくわけて以下の2通りの方法が考えられる。

1. サイコロを東西南北に各面が面した状態で置く置き方は、上面の目（1～6）× 水平に90度ずつ回転（4通り）＝24通りある。この24通りの状態の中で6通りの操作により状態が変化して行くのをトレースする。
2. それぞれの方向を向いている面の目の数を6要素の配列または6個の変数に保持し、6通りの操作により、変数の値を入れ替えながら状態の変化をトレースする。

この問題の場合は各ステップで上面に出ている目の数を加算していくという単純なトレースで、直前のステップまでのサイコロの動きや全体を通しての動きを調べる必要はまったくないので、どちらの方法をとっても結果は同じとなる。しかし、前者の場合は状態間を移動する遷移関数（どの状態からどの操作をするとどの状態になるか）が状態の数（24通り）× 操作の種類（6通り）＝144通りと非常に多くなるのでプログラミングにかかる時間を考慮すると後者を選択するほうが良いだろう。初期配置および各操作による変数の値の入れ替えは以下のようになる。変数名を

上面 vTop, 底面 vBottom, 北面 vNorth, 東面 vEast, 南面 vSouth, 西面 vWest

とし、初期配置を

vTop = 1, vBottom = 6, vNorth = 5, vEast = 3, vSouth = 2, vWest = 4

とする。各操作では2つの面の値がそのまま、残りの4つの面の値が入れ替わる。以下では4つの変数の値を入れ替える際の一時退避用に変数 vTemp を使っている。

問題3 サイコロ (Dice) 続き

解説 (続き)

- **操作 North** vEast と vWest はそのまま。
 $vTemp \leftarrow vTop, vTop \leftarrow vSouth, vSouth \leftarrow vBottom, vBottom \leftarrow vNorth, vNorth \leftarrow vTemp$
- **操作 East** vNorth と vSouth はそのまま。
 $vTemp \leftarrow vTop, vTop \leftarrow vWest, vWest \leftarrow vBottom, vBottom \leftarrow vEast, vEast \leftarrow vTemp$
- **操作 West** vNorth と vSouth はそのまま。
 $vTemp \leftarrow vTop, vTop \leftarrow vEast, vEast \leftarrow vBottom, vBottom \leftarrow vWest, vWest \leftarrow vTemp$
- **操作 South** vEast と vWest はそのまま。
 $vTemp \leftarrow vTop, vTop \leftarrow vNorth, vNorth \leftarrow vBottom, vBottom \leftarrow vSouth, vSouth \leftarrow vTemp$
- **操作 Right** vTop と vBottom はそのまま。
 $vTemp \leftarrow vNorth, vNorth \leftarrow vWest, vWest \leftarrow vSouth, vSouth \leftarrow vEast, vEast \leftarrow vTemp$
- **操作 Left** vTop と vBottom はそのまま。
 $vTemp \leftarrow vNorth, vNorth \leftarrow vEast, vEast \leftarrow vSouth, vSouth \leftarrow vWest, vWest \leftarrow vTemp$

変数間の値の入れ替えを繰り返すため、非常に間違えやすいので、アップロード用の出力データを作成する前に 6 種類の各操作が 1 回だけ行われる入力を自分で作成し、操作後の各変数の値を出力して確認などの動作テストは必須であろう。

原文 : https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/2006-yo-t3-review.html

問題4 コップの移動 (Moving Cups)

配点 20点 競技時間 180分 採点用入力 5組

解説

実際にコップを移動してみるとすぐわかるように、コップを移動させていく方法は単純な一本道である。それゆえ、最も単純な解法は、可能なステップを逐一トレースする（実際にコップを移動させる過程を追ってみること）である。ただし、その場合でも、与えられた初期状態からトレースを開始すると、最初に実行可能なステップが2通りあることがあるので、その両方をトレースして、目的状態に到達するステップ数が少ない方を選ぶことになる。あるいは同じことであるが、すべてのコップがAに重ねられている状態から与えられた初期状態に到達するまでのステップ数と、すべてのコップがCに重ねられている状態から初期状態に到達するまでのステップ数を求めて、それらの小さい方を解としてもよい。

しかし、次のことに気付けば、これらのうち的一方だけをやってみるだけで解は求められる。すべてのコップ (n 枚とす) がAに重ねられている状態から始めて、すべてのコップがCに重ねられるまでに必要な最小ステップ数を $f(n)$ とすると、

1. 1番小さいコップを無視して（無いと思って）、Aにある $n-1$ 個のコップすべてをCに移すことを先ず行う（1番小さいコップの上にはどのコップも重ねることができるので、無いと考えてもよい）。これに必要なステップ数は $f(n-1)$ である。
2. 次に、Aに残した一番小さいコップをBに移動する。
3. Bにある一番小さいコップが無いものと思って、Cにある $n-1$ 個のコップをAに移す。これにかかるステップ数は $f(n-1)$ である。
4. Bにある一番小さいコップをCに移動する。
5. Cにある一番小さいコップが無いものと思って、Aにある $n-1$ 枚のコップをCに移動する。これに必要なステップ数は $f(n-1)$ である。

以上のステップ数を合計すると、

$$f(n) = 3f(n-1) + 2$$

という漸化式が得られる。特に、 $f(1) = 2$ であるから、これを解くと、 $f(n) = 3^n - 1$ であることがわかる。この事実を使うと、Cにすべてのコップが重ねられている状態から与えられた初期状態までのステップ数は、 $(3^n - 1) - (C$ にすべてのコップが重ねられている状態から与えられた初期状態までのステップ数) として求められる。

編集注：上の漸化式を含め、数式・用語は原文の表記を変更せず収録しています。

このように方針が立つと、あとはいかにコップの移動をプログラム上で実装するか（つまり、コップがそれぞれのトレイにある状態をどのように表現するとトレースのシミュレーションが容易になるか）という点だけである。コップはそれまでに積み重ねられている上に重ねて置くしかなく、重ねられた山の一番上のものを取り出すことしか許されていないが、このようなデータの挿入削除の制限を持った構造はスタック (stack) という名前で良く知られているものである。スタックをプログラムで実装する方法を使えば容易にプログラムは書けるが、たとえスタックというものを知らなくても、似たようなことを行うのはむずかしいことではない。

さて実は、このように実際にコップの移動をトレースしてみなくても、ステップ数は式の計算によって求めることもできる。

トレイ A, B, C に重ねられているコップの大きさ（整数値）の集合をそれぞれ A, B, C で表すことにする（集合について知らない人は読み飛ばしてもよい．予選の参加者がこのような解法をすることは出題側では想定していない）．また，初期状態 A, B, C から，トレイ X にすべてのコップを重ねるために必要な最小ステップ数を $f_X(A, B, C)$ で表すことにする（X は A, B, C のいずれか）． $A \cup B \cup C$ 中の最小値を $\min$ で表す．また，空集合を 0 で表す．まず， $f_C(A, B, C)$ は f_A, f_B を使って次のように表すことができる．

最小のコップ $\min$ が C にある場合：

$$f_C(A, B, C) = f_C(A, B, C - \{\min\})$$

最小のコップ $\min$ が B にある場合：

$$f_C(A, B, C) = f_A(A, B - \{\min\}, C) + 1 + f_C(A \cup (B - \{\min\}) \cup C, 0, 0)$$

最小のコップ $\min$ が A にある場合：

$$f_C(A, B, C) = f_C(A - \{\min\}, B, C) + 1 + f_A(0, 0, (A - \{\min\}) \cup B \cup C) + 1 + f_C((A - \{\min\}) \cup B \cup C, 0, 0)$$

この式の 1 行目は，最小のコップが C にある場合，それが無いかのごとく考えてコップをすべて C に移動すればよいことを表している．2 行目は，最小のコップが B にある場合は，その 1 個を無視して，残りをすべて A に集めてから，B に残った最小のコップを C に移動させ，その後で A にあるすべてのコップを（C にある最小のコップの存在は無視して）C に移動させればよいことを表している．3 行目は上述の 1. ～ 5. を表している．

同様に， $f_B(A, B, C), f_A(A, B, C)$ の漸化式が得られる．A, B, C をコップの初期状態とすると，これら3つの漸化式 f_A, f_B, f_C を連立で再帰的に計算し， $f_C(A, B, C)$ と $f_A(A, B, C)$ の小さい方を求める解とすればよい．この式の右辺の集合の要素数は左辺のそれよりも小さくなっているので，計算はコップの総数に比例する時間 $O(|A \cup B \cup C|)$ で終了することがわかる．このとき，上述の $f(n) = 3^n - 1$ を使うと計算はより簡単になる．

なお，繰り返しになるが，この問題は最初に述べたようにステップをトレースする方法で解くことを期待して出題しており，高校生にはむずかしい「式の再帰的計算による解法」で解くことはまったく想定していない．

問題4 コップの移動 (Moving Cups) 続き

解説 (続き)

この問題は「ハノイの塔」と呼ばれる有名な問題をもとにしている。ハノイの塔の問題では、この問題のコップに相当するものの移動はAからCへも、CからAへも直接移動できることになっているが、この問題はそれらを禁止したものである。容易にわかるように、A にすべてのコップがある状態から C にすべてのコップを重ねた状態にするための最小ステップ数はハノイの塔の問題では $2^n - 1$ であり、この問題では上述のように $3^n - 1$ である。この問題の場合、 3^n は可能なコップの配置状態の総数に等しい。A にすべてのコップがある状態から C にすべてのコップがある状態へ移るためには、B にすべてのコップが重ねられている状態を必ず通過し、それは、移動の過程のちょうど真ん中の $(3^n - 1)/2$ ステップ目である。

最後に、別の解法も可能であることも指摘しておく。それは、8クイーン問題とか騎士の周遊問題といった有名問題を解くときに用いられる方法、すなわち、可能なステップの進め方を先へ先へと進めて、失敗したら最小限後戻りして別の可能なステップを先へと進める、という方法である（グラフ理論とアルゴリズムの用語を使って言うと、可能なステップの状態を頂点とし、移動可能なステップ間に辺があるようなグラフを底優先探索することに等しい。通常はそのようなグラフは意識せずに、ステップの進め方を再帰的なアルゴリズムとして記述することが多い）。しかし、この問題の場合、ステップ数が最悪の場合 $3^n - 1$ にもなるので、 n がちょっと大きくなるだけで、それまでにたどって来た道筋を（失敗したときに後戻り [バックトラッキング (backtracking) という] できるように）記憶しておく必要がある [再帰的プログラムの場合には、再帰が発生するたびにそのときの関数の局所状態がメモリ内にシステムによって自動的に記憶される]）ために、メモリが実行時にオーバーフローしてしまうという決定的な欠点がある。

さらに、別の解法として、どの状態からも「A から B」「B から A」「B から C」「C から B」の4通りしかコップの移動方法がないので、 m 桁の4進数（ m ステップ分のコップの移動を表す）すべてを系統的に生成して、そのうちで実際にコップの移動に対応しているものを求めるという解法もある（全数検査法）。しかし、この方法では、 4^m 通りの場合すべてを試さなければならないので、ごく小さい m にしか対応できないという致命的欠点がある。それでも、全数検査法で解けば4点（手で計算することも可能）、再帰で解けば12点取れるように入力データを決めて出題した。また、 n およびそれに対応して決まる m の値の上限は、最小ステップ数が32ビット数で表現できる範囲内に抑えた。

原文：https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/2006-yo-t4-review.html

問題5 カードの山 (Card Piles)

配点 20点 競技時間 180分 採点用入力 5組

解説

この問題は、模擬試験2の5番と同様に、20点を取るためには数学的センスが必要な問題である。IOIの本番の問題を解くためには、高いプログラミングの技能が必要なことは言うまでもないが、数学的な思考力（特に、組合せや確率についての知識）も必須である。また、試験の解答に当たって、解を予測するためにシミュレーションを行なうことの必要性や有効性を知ってもらいたいという趣旨も込めて出題した。

しかし、プログラミング技能だけに頼った力ずくでも3つの入力ファイルに対しては答えが求められるように、また、上述のような解の予測により4つ目の入力ファイルに対しても答えが求められるように入力データを作ったので、はじめに、その方法について述べよう。

nk 枚のカードの配り方は全部で $(nk)!$ 通りあるから、これらすべての場合（ $(nk)!$ 通りの順列）を生成して、実際にゲームを行えば確率は正確に求めることができる。順列すべてを生成する方法はいろいろあるが、ここでは省略する。 $(nk)!$ の値が 1000000 未満である n, k の値は以下の通りである。1000000 未満だと 1 秒以内で実行できる。

n	k = 1	2	3	4	5
3	6	90	1680	34650	756756
4	24	2520	369600		
5	120	113400			
6	720				
7	5040				
8	40320				
9	362880				

入力データのうち2つは、このような全数検査で答えが求められるものとした。

さて、まず全数検査を行う簡単なプログラムを作って、いろんな n, k の値について確率を求めてみると、 $m = 0$ なら k の値とは無関係に、求める確率が $1/n$ になるであろうことが推測できる（ $m = 1$ の場合には、このような推測をすることはむずかしい）。冒頭に書いたように、このようなことができる能力も実戦力として有効である。この推測ができれば、5つの入力ファイルのうち3つまでは正しい答えが求められる。

問題5 カードの山 (Card Piles) 続き

解説 (続き)

$m = 1$ の場合の答えを求めるには、数学的に確率を求めることが必要である。その確率は $1/n + 1/n(1 + 1/2 + \cdots + 1/(n-1))$ である (証明は後述)。このような理論値が求められれば、あとは計算するだけであるが、小数第 10000 桁まで求めることを要求されているので、配列を用いた多倍長計算をするしかない。すなわち、配列要素 $a[0], a[1], a[2], \dots, a[10005]$ で小数

$$a[0] . a[1]a[2]\cdots a[10005]$$

を表して ($a[0]$ は整数部を, $a[i]$ は小数第 i 位を表す: $a[i] = 0, 1, 2, \dots, 9$) , 筆算で $1/n$ を計算したり桁数の多い足し算を計算したりするのと同様に計算する。つまり、模擬試験 2 の問題 5 と同様であるが、今回注意しなければならないのは、誤差の問題である。要求されているのは最大 10000 桁まで求めることであるが、やや多目の桁数を取って計算する必要がある。というのは、足し算を n 回行うので、末尾の桁における誤差 (10000 回の足し算なら影響が 5 桁に及ぶ) の影響を排除するためである。なお、 $a[i]$ を 10 進 1 桁に限定しないほうが計算時間は短くなることに注意する。ただし、 $n = 10000$ 以下なら (この問題の入力データは $n \leq 10000$ としてある) , そこまでしなくても時間がかからずに計算できる。

最後に、カードをランダムに配ってゲームを 10000 回程度試行してみて、答えを予測するという手法も考えられるが、問題を作るに当たって実際に乱数を生成して試したところ、 $m = 0$ の場合の解の推測はかろうじてできる可能性があるものの、 $m = 1$ の場合は無理のようである。5 つの入力データのうちの 2 つは、このような方法でも求められるものとした。

問題5 カードの山 (Card Piles) 続き

解説 (続き)

2回目までに成功する確率が $1/n + 1/n (1 + 1/2 + 1/3 + \dots + 1/(n-1))$ であることの証明:

どの山にどのような数 ($1 \sim n$) が積まれているかではなく、順次山をたどる順番 (すなわち, $(1, 2, \dots, n) \times k$ の順列, がどうであるかが本質的なことである. 1 回目の試行では, n 枚目が 1 (= 最初に引いた山の番号) であることが成功するための必要十分条件である (k 枚目の 1 が出るまでは, どの山も十分な枚数が残っている). 最後に 1 が出現する確率は $1/n$ である (kn 枚目の数として 1 は k 通りの選び方があり, そのそれぞれについてそれ以外の $k(n-1)$ 枚はそれ以前に自由に現れてよいから, 最後に 1 が来る場合の数は $k(kn-1)!$ である. よって, 最後に 1 が来る確率は $k(kn-1)! / (kn)! = 1/n$ である). このように, k は確率に関与しない.

さて, 1 回目で失敗したとき, 2 回目で成功する確率を考えよう. $k=1$ の場合を考えれば十分である. 1 回目の試行で現れた数の順列が

1 - - - $\dots$ - - の場合には山が 1 つ無くなっていて,
 - 1 - - $\dots$ - - の場合には山が 2 つ無くなっていて,
 - - 1 - $\dots$ - - の場合には山が 3 つ無くなっていて,
 . . .

といったようになることは明らかである. それぞれの場合は確率 $1/n$ (1 が順列の特定の位置に来る確率) で起こる. しかも, 山が i 個無くなっていった場合には ($1 \leq i \leq n-1$), そのあとで 2 回目の試行をしたときに成功する確率は $1/(n-i)$ である. これらの事象は排反独立であるから, 2 回目に成功する確率 $1/n (1 + 1/2 + \dots + 1/(n-1))$ が得られる. 1 回目で成功する確率を加えて, 2 回目までに成功する確率は $1/n + 1/n (1 + 1/2 + \dots + 1/(n-1))$ である.

原文: https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/2006-yo-t5-review.html

出典: 情報オリンピック日本委員会「第5回日本情報オリンピック JOI 2005/2006 予選競技課題」. 原資料は CC BY-SA 4.0 で提供されています. 本 PDF は各解説ページを問題順に再構成したものです. 原資料に問題名がないため, 日本語表題と英訳は内容に基づく便宜的な見出しです.
https://www2.ioi-jp.org/joi/2005/2006-yo-prob_and_sol/index.html
https://www.ioi-jp.org/problem_archive